

CMSC 3613

Programming Assignment: Dijkstra's Algorithm

Due Date: Check the D2L calendar for the due date.

Assignment:

The task of this project is to determine the shortest/cheapest flight plans to travel between cities by an airline, using Dijkstra's algorithm.

The airline map is abstracted as a digraph, and is further represented as an adjacency matrix, recorded in the first input file. The format of this digraph file is ([input1.dat](#)):

```
6
S A B C D T
0 1 5 INF INF INF
1 0 INF 2 3 INF
5 INF 0 1 INF 3
INF 2 1 0 3 4
INF 3 INF 3 0 2
INF INF 3 4 2 0
```

{The first line is the number of cities.

The second line is the list of the names for those cities.

The rest part is the adjacency matrix. If there is no connection between two cities, the weight is specified as "INF", which means infinity. If there is a connection, we can use an integer to specify the weight.}

There is another input file to provide the queries. The format of this query file is ([query1.dat](#)):

```
S T
S D
```

{Each line is a query, consists of a source and a destination.}

The output file provides the shortest distances for the queries. The format of this output file is:

```
6
4
```

{Each line is the result for each query.}

The input and output have been handled by the given program as a reference for you. Similar as the shortest distance, we can also consider the shortest time and cheapest price for the flight plan. To specify both time and cost, we can change the first file for the adjacency matrix as follow ([input2.dat](#)):

```
4
Austin Chicago Dallas Houston
0|0 144|192 INF|INF 95|39
155|200 0|0 INF|INF INF|INF
INF|INF INF|INF 0|0 101|51
INF|INF INF|INF 100|50 0|0
```

{This is very similar as the digraph file above. But each element in the adjacency matrix is changed. The format of each element is: time|cost.}

And in the query file, in each line, we need to add an additional field to specify the mode ([query2.dat](#)):

```
Dallas Austin T
Chicago Dallas C
```

{This is very similar as the query file above. But at the end of each line, we add a “T” or a “C” to show that the query is targeting the shortest time (T) or cheapest price (C).}

It’s possible that there is no flight to connect two cities, and in this case, we can output “No available path.” for that query:

```
No available path.
289
```

Please note:

The provided program is only compatible with input1.dat and query1.dat, and your task is to modify the provided code so that it could work with input2.dat and query2.dat (and no need to be compatible to input1.dat and query1.dat any more) and provide either the shortest time or cheapest price according to the queries

Extra credits (extra 15%):

In addition to the shortest time or cheapest price, output the actually path associated with the shortest time or cheapest price:

```
No available path.
Chicago Austin Houston Dallas 289
```

Requirements:

1. Your program should follow the instructions described above in the “Assignment” section.
2. Use C++ language for implementation. The input and output formats hen been defined above, please follow the specifications.
3. The command to invoke the program should be like:

./p06 input2.dat query2.dat output2.dat

p06 should be the name of your executable (p06 should be a fixed name), and other arguments are files for input, query and output (can be changed).

Evaluation:

This project will be evaluated according to the correctness of the various methods specified above, and secondarily according to the quality of your code. The rubric is as follows:

Categories	Weights
read input	35%
get the plan: shortest time or cheapest price	40%
correctness	15%
get the real path	15% (bonus)
report	10%

Submission:

1. Please provide a readme file, to help with the compilation and execution of your code. For example, information about your operating system and detailed command to compile your code should be included.

2. You need to submit a report by the due date on D2L. The report should include the following items:
 - 1) Your name and UCO email address
 - 2) The project number, i.e., p06
 - 3) A brief discussion of your implementation: just like an explanation of your idea in an interview.
Another discussion: in this implementation, the graph is represented by an adjacency matrix, and the vertex is in int type. But what if (1) we want to design a new class type for the Vertex to include the name of the city and (2) we want to design a new class type for the Weight to include both the time and cost, then how would you like to represent the graph? Please briefly explain your ideas, and no real code is required.
 - 4) Screenshots of a test run.
3. Your source code also needs to be submitted.
4. All the files need to be zipped as **p06_group*.zip**, where * means your group number, e.g., the submission from group 3 should be named as p06_group3.zip.

Notes:

1. To be considered on time, the program must be turned in by the due date.
2. Programs must reflect your knowledge and work, not others. You may ask others questions about algorithms, methods and programming style, but when you start writing code, you must work only with your group member(s).
3. No point, zero (0), will be given to any program containing compilation errors.